

Restoring M1/M8 Numerical Agreement in RDNA4 Speculative Decoding

Terrydaktal · 17 September 2026 · Technical report, version 4

Abstract

A pinned Radiance/vLLM inference stack produced different target logits when the same saved token history was processed one position at a time (M1) or eight positions at a time under D7 speculative verification (M8). The investigation localized numerical differences in GDN prefill and decode, convolution, gated normalization, attention, residual normalization and the BF16 vocabulary head. The repaired, compiled implementations now agree on top-1/10/20 membership, ordering, retained scores, inclusive boundary ties and full-vocabulary logit hashes at 10,000 decode positions and 23 initial-prefill predictions. Four subsequent performance changes reduced a repaired verification round from approximately 76 ms to 60 ms in the measured 60K-input control.

A later eager/compiled investigation isolated intermediate BF16 rounding and native RoPE multiplication differences. Aligning both produced identical full logit vectors at 320 decode positions and the final prefill prediction, with identical captured inputs and outputs at all 465 matched activation boundaries. This newer eager/compiled M8 result uses a separate 320-position replay; the 10K comparison has not been repeated under the new rounding settings.

This report documents an empirical result on a pinned configuration. It does not establish arbitrary-input equivalence, equality to an independently specified mathematical model, or improved task accuracy. Historical mode differences, their diagnosis and the remaining coverage limits are distinguished below.

1. Configuration and reference

- One AMD Radeon AI PRO R9700, gfx1201, TP1.
- Radiance 1.0.16 / vLLM 0.28, PyTorch 2.12.0, ROCm 7.14, Triton 3.7.1.
- Qwen3.8-27B-Uncensored-MXFP4-awq; 64 layers, including 48 GDN and 16 attention layers; hidden width 5,120; vocabulary size 248,320.
- The target vocabulary head uses the full original BF16 weights in both arms. INT2 candidate selection is excluded from the equality claim. Other model weights, activation quantization and cache formats remain part of the pinned quantized backend; this is not an unquantized BF16-model comparison.
- The compiled comparisons use Inductor and piecewise GPU graphs, with capture sizes [1, 2, 4, 8]. Actual target graph replay is audited.
- The numerical reference is the explicitly selected serial implementation. Repairs also affect M1/preparation paths, so repaired M1 is measured afresh.

The 10K corpus comprises 23 saved Pi continuations with 57,008–65,527-token prefixes. Both arms consume identical saved token histories at aligned logical positions. Each M8 group processes eight positions with all seven supplied proposals accepted. The denominator is 10,000 processed decode positions, excluding intermediate-prefill head calls. Final initial-prefill predictions have a separate denominator of 23. No generated tools execute during replay.

2. Agreement definitions

For each position t , let $z1(t)$ and $z8(t)$ be the full vocabulary logit vectors. T_k sorts logits by decreasing score and then increasing token ID for ties.

```
set agreement(k)  = count(set(T_k(z1)) == set(T_k(z8))) / N
order agreement(k) = count(T_k(z1) == T_k(z8)) / N
mean shared(k)    = sum(|set(T_k(z1)) intersect set(T_k(z8))|) / N
```

Retained-score equality, inclusive boundary-tie membership and full-vector SHA-256 equality are separate checks. Matching hashes are strong empirical evidence, not a collision-free mathematical proof. Agreement measures execution consistency; it is not a percentage of questions answered correctly.

3. Top-1/10/20 and ordering results

Final compiled, optimized repair: full 10K corpus

Prediction	Same token set	Same ordering	Mean shared tokens
Top 1	10,000 / 10,000 (100%)	10,000 / 10,000 (100%)	1 / 1
Top 10	10,000 / 10,000 (100%)	10,000 / 10,000 (100%)	10 / 10
Top 20	10,000 / 10,000 (100%)	10,000 / 10,000 (100%)	20 / 20

Retained scores, inclusive tie sets and full-vocabulary hashes also match at all 10,000 positions. All measures separately match for **23/23 initial-prefill predictions**. Both workers exited successfully. The completion audit checked all per-continuation receipts, fixture identities, widths, aligned positions, runtime receipts and the recomputed aggregate.

Compiled before/after control: 320 positions

This is the directly comparable compiled control, separate from the larger historical eager run. Each implementation is compared with its corresponding M1 reference.

Prediction	Original M8: same set	Original M8: same order	Fixed optimized M8: same set	Fixed optimized M8: same order
Top 1	318/320 (99.375%)	318/320 (99.375%)	320/320 (100%)	320/320 (100%)
Top 10	161/320 (50.3125%)	17/320 (5.3125%)	320/320 (100%)	320/320 (100%)
Top 20	76/320 (23.75%)	0/320 (0%)	320/320 (100%)	320/320 (100%)

Full-vocabulary hashes match at 0/320 original positions and 320/320 repaired positions. The original initial-prefill vectors also differ. Original compiled M1 versus repaired compiled M1 agrees on top-1 at 319/320 positions and differs in every full-vector hash; reference changes are material to this experiment.

A [fresh repaired compiled M1/M8 replay](#) also matches every complete output vector and the final prefill vector. Its [per-stage activation comparison](#) matches outputs at all 466 named boundaries; captured inputs match at 465, with the remaining input unobserved. Serial positions are aligned with each M8 group, and physical cache capacities are recorded separately in the [admission receipt](#).

Historical unfixed eager baseline: same 10K corpus

Prediction	Same token set	Same ordering	Mean shared tokens
Top 1	9,852/10,000 (98.52%)	9,852/10,000 (98.52%)	0.9852 / 1
Top 10	4,857/10,000 (48.57%)	799/10,000 (7.99%)	9.3487 / 10
Top 20	2,270/10,000 (22.70%)	9/10,000 (0.09%)	18.6781 / 20

These historical results use eager execution. They must not be presented as an unfixed **compiled** 10K measurement. The final compiled run uses a fresh compiled M1 reference, not this saved eager reference.

4. Compiled stage timings and isolated correctness measurements

Both timing profiles use the same 60,000-input-token Pi fixture and the optimized compiled configuration: **enforce_eager=false, Inductor, PIECEWISE GPU graphs, capture sizes [1, 2, 4, 8]**. The actual traces contain **65 target GPU graph launches in every captured round**, in both arms. Runtime receipts are recorded in [compiled-execution-audit.json](#). These are not eager timings. The full BF16 target head is used on both sides.

The finer audit found incomplete expected target-kernel inventories in old round 8 and fixed round 7 (one-based): respectively two and three missing recorded events relative to the modal inventory. This does not establish that the backend skipped those computations. The table below uses the **six paired rounds with complete inventories on both sides**, rounds 1–6. Selection depends on event counts, not speed. Both original eight-round receipts and the [original aggregate table](#) are retained; all recorded dispatches, including excluded rounds, remain in the evidence.

Each time is the sum of recorded GPU dispatch durations divided by six. Layer stages sum all their layer instances in a round. The 256 projection dispatches per round have been mapped using the checked four-projection sequence in each of 64 layers

and the intervening GDN, attention and SiLU markers. The roughly 25 ms folded-projection total is **the joint MLP gate/up projection across all 64 layers**, rather than an unexplained collection of unrelated stages.

The isolated columns compare **compiled M8 against compiled M1** on 320 positions. Each stage receives common reference inputs and state; only that stage is substituted, then the reference remainder produces vocabulary logits. The columns report vocabulary top-20 set/order agreement, not the largest coordinates of an intermediate activation. “No correctness repair” describes the code change; unmeasured stages remain labeled as such.

The **compiled capture** reproduces all 320 release output vectors and the final prefill vector. Every isolated reference remainder is checked against those saved outputs before counting a result. Graph replay is enabled in the isolated kernel comparisons.

The **head** and **final normalization** checks supply the completed table cells. In both, full reference logits match at **0/320 old and 320/320 fixed** positions, despite the head retaining the same top-20 rankings. The final-norm cut includes its fused retained-residual addition: the original kernel omits an intermediate BF16 rounding of that sum. Its result does not stand in for other normalization layers. Input/weight immutability and an injected bit error are checked.

Compiled stage	Correctness fix?	Old M8 ms	Fixed M8 ms	Change ms	Old compiled M8 vs compiled M1 Isolated top-20 set/ order	Fixed compiled M8 vs compiled M1 Isolated top-20 set/ order	Timing explanation
Embedding + first input normalization	Normalization repair; embedding unchanged	0.004	0.008	+0.004	Not yet measured	Not yet measured	Preserve the reference normalization rounding; the original embedding/norm fusion is indivisible.
Layer input residual/ normalization	Correctness + performance	0.180	0.357	+0.177	Not yet measured	Not yet measured	Preserve reduction and rounding; retain FP32 residual sums in registers.
GDN input activation FP8 quantization	No correctness repair	0.487	0.128	-0.359	Not yet measured	Not yet measured	Same quantization kernel and call count; the measured reduction has no isolated causal attribution.
GDN input projection	No correctness repair	3.879	3.875	-0.004	Not yet measured	Not yet measured	Unchanged MXFP4 projection arithmetic; timing differences are observations, not a projection optimization.
GDN layout/copies and buffer initialization	Performance only	0.762	0.332	-0.430	Not yet measured	Not yet measured	Preserve packed QKV views; remove split materializations and repacking. Remaining copies are included.
GDN convolution	Correctness + performance	0.772	0.228	-0.543	Not yet measured	Not yet measured	Match serial product/accumulation order and rolling history; packed transport removes surrounding copies.
GDN recurrence and gates	Correctness repair	1.150	1.218	+0.067	Not yet measured	Not yet measured	Match gate precision, reduction order; recurrent-state transition and output rounding.
GDN output gated normalization	Correctness + performance	0.399	0.098	-0.301	Not yet measured	Not yet measured	Keep the serial row tile while processing independent rows concurrently; original fused constituents remain grouped.
GDN output activation FP8 quantization	No correctness repair	0.254	0.131	-0.123	Not yet measured	Not yet measured	Same quantization kernel and call count; the timing reduction is not an established quantizer improvement.
GDN output projection	No correctness repair	1.897	1.886	-0.011	Not yet measured	Not yet measured	Unchanged MXFP4 arithmetic; no causal speedup claimed.
Attention input activation FP8 quantization	No correctness repair	0.167	0.042	-0.124	Not yet measured	Not yet measured	Same quantization kernel and call count; timing cause is not isolated.

Compiled stage	Correctness fix?	Old M8 ms	Fixed M8 ms	Change ms	Old compiled M8 vs compiled M1 Isolated top-20 set/ order	Fixed compiled M8 vs compiled M1 Isolated top-20 set/ order	Timing explanation
Attention input projection	No correctness repair	1.104	1.099	-0.004	Not yet measured	Not yet measured	Unchanged QKV MXFP4 projection; no causal speedup claimed.
Attention Q/K normalization, RoPE and layout	Normalization repair; RoPE unchanged	0.220	0.240	+0.020	Not yet measured	Not yet measured	Keep serial normalization arithmetic. Fused original normalization/RoPE constituents cannot be timed separately.
Attention KV write	No correctness repair	0.157	0.047	-0.110	Not yet measured	Not yet measured	Same cache-write kernel and KV format; timing cause is not isolated.
Attention decode	Correctness + performance	4.112	5.262	+1.150	Not yet measured	Not yet measured	Preserve each query's causal tile/softmax decisions. Share KV reads within one or two tile-aligned query groups; two groups repeat context work.
Attention split-KV merge	Correctness repair	0.253	0.121	-0.132	Not yet measured	Not yet measured	Use each query's serial split/merge arithmetic; the observed reduction has not been isolated from the decode change.
Attention output gating	No correctness repair	0.039	0.032	-0.006	Not yet measured	Not yet measured	Unchanged pointwise operation; measured variation has no isolated causal attribution.
Attention output activation FP8 quantization	No correctness repair	0.167	0.046	-0.121	Not yet measured	Not yet measured	Same quantization kernel and call count; timing cause is not isolated.
Attention output projection	No correctness repair	0.562	0.521	-0.041	Not yet measured	Not yet measured	Unchanged output MXFP4 projection; no causal speedup claimed.
Post-attention/ GDN residual/ normalization	Correctness + performance	0.161	0.353	+0.192	Not yet measured	Not yet measured	Preserve serial reduction/rounding and keep residual values in registers.
MLP gate/up input FP8 quantization	No correctness repair	0.648	0.168	-0.481	Not yet measured	Not yet measured	Same quantization kernel and call count; timing cause is not isolated.
MLP gate/up projection	No correctness repair	24.336	25.500	+1.164	Not yet measured	Not yet measured	One joint gate/up GEMM per layer, 64 per round. The per-layer table subdivides this total; gate and up have no separate measured durations.
MLP SiLU and gating	No correctness repair	0.148	0.179	+0.031	Not yet measured	Not yet measured	Unchanged compiler-fused SiLU/gating; no causal timing improvement claimed.
MLP down input FP8 quantization	No correctness repair	0.713	0.298	-0.415	Not yet measured	Not yet measured	Same quantization kernel and call count; timing cause is not isolated.
MLP down projection	No correctness repair	5.174	5.147	-0.027	Not yet measured	Not yet measured	Unchanged MXFP4 down projection; no causal speedup claimed.
Final normalization/ layout	Correctness repair	0.003	0.006	+0.003	306/320; 230/320	320/320; 320/320	Preserve final-normalization reduction and the BF16 rounding of its retained residual sum; isolated check includes that fused addition.

Compiled stage	Correctness fix?	Old M8 ms	Fixed M8 ms	Change ms	Old compiled M8 vs compiled M1 Isolated top-20 set/ order	Fixed compiled M8 vs compiled M1 Isolated top-20 set/ order	Timing explanation
Full BF16 target head	Correctness + performance	4.101	4.134	+0.033	320/320; 320/320	320/320; 320/320	Interleave two arithmetic-preserving M4 groups in one HIP launch, removing duplicated launch and concatenation overhead.
Drafter	No target correctness repair	6.332	6.404	+0.072	N/A: not a target prediction stage	N/A: not a target prediction stage	Unchanged proposal model; its timings and predictions are not target-M1 equivalence measurements.
Other GPU bookkeeping	Not an isolated numerical stage	0.935	0.549	-0.385	N/A: not a target prediction stage	N/A: not a target prediction stage	Sampling/state bookkeeping outside the model scopes. Exact semantic attribution is unavailable; each kernel remains listed below.

Target-body subtotal: **47.746 ms old; 47.323 ms fixed**. Total recorded GPU kernel duration: **59.114 ms old; 58.410 ms fixed**. These are subtotals, not extra stages. Kernel durations can overlap and exclude host/queue gaps; they are not wall-clock round latency.

Timing changes that are not yet causally isolated

Activation quantization still uses the same named kernel and **256 calls per round** in each arm. Across the six complete rounds its sum is **2.437 ms old versus 0.814 ms fixed**. The traces establish the difference, not its cause. There was no quantizer replacement or removal. Execution context, layouts, cache behavior and measurement effects have not been separated by an isolated quantizer experiment, so the report does not claim a threefold quantizer fix.

Attention decode averages **4.112 ms old versus 5.262 ms fixed**, but the fixed per-round totals are **3.546, 7.054, 3.570, 6.916, 3.588 and 6.901 ms**. The repaired source forms one or two query groups according to the eight queries' 16-token tile boundaries, preserving each query's M1 causal/softmax decisions. A second active group repeats cached-context work. This is a plausible mechanism for the two timing bands, not a measured attribution of every slow round; the original traces do not record the kernel's sequence-length argument. The split-KV merge is separate: **0.253 ms old versus 0.121 ms fixed**.

Unchanged projection timings also vary. A timing delta by itself does not identify a correctness repair, an optimization, or its causal benefit.

Every layer, including the 64 contributions to the gate/up total

Layer numbers are zero-based. "Other three projections" means the input, attention/GDN output, and MLP down projections; all individual semantic stage values are also retained in stage-times.json. Non-projection work includes normalization, gating, quantization, attention/GDN operations and copies.

Expand all 64 layers

Layer (zero-based)	Type	All layer work old ms	Fixed ms	Gate/up old ms	Fixed ms	Other three projections old ms	Fixed ms	Non-projection old ms	Fixed ms
0	GDN	0.6277	0.6242	0.3708	0.3665	0.1946	0.1939	0.0623	0.0639
1	GDN	0.6265	0.6256	0.3701	0.3673	0.1987	0.1981	0.0577	0.0602
2	GDN	0.6293	0.6325	0.3691	0.3711	0.1995	0.2001	0.0606	0.0613
3	Attention	0.8354	0.9308	0.3667	0.3804	0.1825	0.1801	0.2862	0.3702
4	GDN	0.6226	0.6506	0.3674	0.3887	0.1982	0.1991	0.0570	0.0628
5	GDN	0.6251	0.6507	0.3670	0.3898	0.1992	0.1983	0.0589	0.0626
6	GDN	0.6277	0.6572	0.3680	0.3931	0.1990	0.1998	0.0607	0.0644
7	Attention	0.8440	0.9542	0.3723	0.3970	0.1822	0.1809	0.2895	0.3763

Layer (zero-based)	Type	All layer work old ms	Fixed ms	Gate/up old ms	Fixed ms	Other three projections old ms	Fixed ms	Non-projection old ms	Fixed ms
8	GDN	0.6286	0.6599	0.3716	0.3954	0.1984	0.1998	0.0585	0.0647
9	GDN	0.6357	0.6609	0.3768	0.3964	0.1999	0.2000	0.0590	0.0645
10	GDN	0.6412	0.6601	0.3794	0.3949	0.1993	0.2002	0.0625	0.0650
11	Attention	0.8539	0.9605	0.3767	0.3956	0.1819	0.1802	0.2953	0.3847
12	GDN	0.6375	0.6587	0.3784	0.3942	0.1992	0.2001	0.0599	0.0645
13	GDN	0.6413	0.6617	0.3827	0.3968	0.1992	0.2003	0.0595	0.0646
14	GDN	0.6440	0.6602	0.3806	0.3950	0.2006	0.2002	0.0629	0.0650
15	Attention	0.8654	0.9529	0.3845	0.3946	0.1832	0.1810	0.2978	0.3774
16	GDN	0.6461	0.6569	0.3866	0.3920	0.1995	0.1998	0.0600	0.0652
17	GDN	0.6470	0.6576	0.3864	0.3923	0.2002	0.2003	0.0603	0.0650
18	GDN	0.6535	0.6600	0.3878	0.3946	0.2012	0.1998	0.0645	0.0656
19	Attention	0.8723	0.9618	0.3892	0.3968	0.1825	0.1807	0.3006	0.3843
20	GDN	0.6500	0.6640	0.3876	0.3961	0.2014	0.2007	0.0609	0.0672
21	GDN	0.6486	0.6650	0.3871	0.3975	0.2009	0.2016	0.0606	0.0659
22	GDN	0.6676	0.6654	0.3893	0.3983	0.2002	0.2013	0.0781	0.0658
23	Attention	0.8889	0.9671	0.3887	0.3988	0.1840	0.1806	0.3161	0.3877
24	GDN	0.6767	0.6630	0.3873	0.3974	0.2002	0.2007	0.0891	0.0648
25	GDN	0.7025	0.6643	0.3874	0.3977	0.2022	0.2015	0.1129	0.0651
26	GDN	0.7267	0.6669	0.3875	0.3991	0.2030	0.2011	0.1362	0.0667
27	Attention	0.9473	0.9651	0.3861	0.4007	0.1859	0.1812	0.3752	0.3832
28	GDN	0.7142	0.6682	0.3858	0.4001	0.2040	0.2013	0.1243	0.0667
29	GDN	0.7228	0.6690	0.3861	0.4014	0.2018	0.2012	0.1349	0.0664
30	GDN	0.7368	0.6691	0.3840	0.4023	0.2029	0.2008	0.1498	0.0660
31	Attention	0.9463	0.9633	0.3805	0.3998	0.1855	0.1812	0.3803	0.3823
32	GDN	0.7167	0.6694	0.3805	0.4011	0.2017	0.2006	0.1345	0.0677
33	GDN	0.7185	0.6682	0.3804	0.4010	0.2027	0.2011	0.1354	0.0661
34	GDN	0.7366	0.6673	0.3831	0.4000	0.2024	0.2000	0.1511	0.0673
35	Attention	0.9570	0.9649	0.3842	0.4005	0.1859	0.1816	0.3870	0.3827
36	GDN	0.7207	0.6674	0.3840	0.4012	0.2033	0.1999	0.1334	0.0664
37	GDN	0.7206	0.6703	0.3826	0.4033	0.2037	0.2010	0.1343	0.0660
38	GDN	0.7334	0.6721	0.3803	0.4047	0.2032	0.2010	0.1499	0.0663
39	Attention	0.9434	0.9633	0.3761	0.4020	0.1846	0.1814	0.3827	0.3799
40	GDN	0.7112	0.6684	0.3761	0.4005	0.2020	0.2023	0.1330	0.0657
41	GDN	0.7139	0.6690	0.3778	0.4005	0.2023	0.2022	0.1338	0.0662
42	GDN	0.7295	0.6711	0.3793	0.4024	0.2016	0.2021	0.1486	0.0666
43	Attention	0.9510	0.9615	0.3789	0.4022	0.1857	0.1816	0.3864	0.3777
44	GDN	0.7158	0.6715	0.3788	0.4039	0.2032	0.2013	0.1337	0.0662
45	GDN	0.7162	0.6716	0.3798	0.4052	0.2024	0.2004	0.1340	0.0661

Layer (zero-based)	Type	All layer work old ms	Fixed ms	Gate/up old ms	Fixed ms	Other three projections old ms	Fixed ms	Non-projection old ms	Fixed ms
46	GDN	0.7300	0.6754	0.3797	0.4057	0.2016	0.2028	0.1487	0.0669
47	Attention	0.9493	0.9824	0.3813	0.4072	0.1854	0.1814	0.3825	0.3939
48	GDN	0.7205	0.6762	0.3842	0.4060	0.2025	0.2018	0.1338	0.0683
49	GDN	0.7241	0.6725	0.3863	0.4058	0.2029	0.2005	0.1349	0.0662
50	GDN	0.7375	0.6748	0.3860	0.4067	0.2021	0.2010	0.1494	0.0672
51	Attention	0.9484	0.9750	0.3815	0.4048	0.1857	0.1820	0.3811	0.3883
52	GDN	0.7172	0.6680	0.3806	0.4011	0.2028	0.2008	0.1338	0.0661
53	GDN	0.7171	0.6687	0.3820	0.4013	0.2028	0.2018	0.1324	0.0656
54	GDN	0.7346	0.6712	0.3825	0.4036	0.2021	0.2016	0.1501	0.0659
55	Attention	0.9491	0.9732	0.3796	0.4018	0.1858	0.1811	0.3837	0.3903
56	GDN	0.7118	0.6712	0.3768	0.4025	0.2020	0.2025	0.1330	0.0662
57	GDN	0.7120	0.6739	0.3764	0.4054	0.2022	0.2017	0.1334	0.0667
58	GDN	0.7273	0.6730	0.3767	0.4052	0.2017	0.2007	0.1489	0.0671
59	Attention	0.9448	0.9702	0.3780	0.4043	0.1861	0.1819	0.3807	0.3840
60	GDN	0.7139	0.6724	0.3782	0.4065	0.2026	0.2000	0.1330	0.0658
61	GDN	0.7141	0.6765	0.3784	0.4076	0.2022	0.2016	0.1335	0.0673
62	GDN	0.7313	0.6763	0.3791	0.4066	0.2017	0.2007	0.1504	0.0690
63	Attention	0.9430	0.9824	0.3772	0.4079	0.1861	0.1816	0.3796	0.3928

Every recorded compiled kernel within each semantic stage

Calls are totals over the six retained rounds; times are per-round means. Compiler-fused operations remain indivisible. The tables do not invent times for separate constituents of a single GPU dispatch. Individual dispatch durations, layer IDs and round IDs are retained in the two evidence/*-compiled-dispatches.json exports; they contain no token IDs, raw activations, absolute timestamps or process identifiers.

Expand the complete kernel breakdown

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Fixed ms per round
drafter / Drafter Cijk_Alik_Blj_BBS_BH_Bias_HA_S_SAV_UserArgs_MT16x16x32_MI16x16x1_SN_LDSB1_AFC1_AG0_AGGSUA0_AGNTAB0_AFEM1_AFEM1_ASEM1_CD1_1_CLR0_CLS0_CADS0_DTLA0_DTLB0_DTLMO_DTVAO_DT VB1_DTVMSA0_DTVMSB0_DTVSM0_DPLB0_EPS0_ELFLR0_EMLLn1_FDSI0_GRP1_GRVWA8_GRVWB8_GSU AMB_GLS0_HPLR0_ISA1201_ICIW0_IU1_K1_LDSTI0_LBSPPA128_LBSPPB0_LBSPPMSA0_LBSPPMSB0_ LBSPPM0_LPA16_LPB0_LPMXA0_LPMXB0_LPM0_LRVW8_LWPMn1_MIAV1_MIWT1_1_MXLIBL_MXSFNS_MO 40_MGRIPM1_NTN1_NTA0 NTB0_NTC0_NTD0_NTE0_NTMXA0_NTMXB0_NTM0_NTWS0_NVn1_NVA0_NVBO_ NVC0_NVD0_NVE0_NVMXA0_NVMXB0_NVM0_NVWS0_NEPBS0_NLCA1_NLCB2_ONLL1_PAP0_PGL0_PGR1_P LR1_PKA0_SGROB0_SIA3_SS0_SPO0_SRVW0_SSO0_SVW8_SK0_SKFTR0_SKFDPO0_SKXCCM0_SNLL0_SIP1 _SGRO0_TDMI0_TDMIM0_TDMS0_TIN0_THn1_THA0_THB0_THC0_THD0_THE0_THMXA0_THMXB0_THM0_T HWS0_TLDS1_TLDSM1_ULSGRO0_USL1_USLMX0_UIOFGRO0_UPLRP0_USFGRON1_USI0_VSn1_VWA1_VWB1_ WSGRA0_WSGRB0_WS32_WG16_2_1.kd	6 / 6	0.007146	0.007072

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Fixed ms per round
drafter / Drafter Cijk_Alik_Bljk_BBS_BH_Bias_HA_S_SAV_UserArgs_MT16x32x256_MI16x16x1_SN_LDSB0_AFC1_AG 0_AGGSUA0_AGNTAB0_AFEM1_AFEM1_ASEM1_CD1_1_CLR1_CLS0_CADS0_DTLA0_DTLB0_DTLMO_DTVA0_D TVB1_DTVMXSA0_DTVMSB0_DTVSM0_DPLB0_EPS1_ELFLR0_EMLLn1_FDSI0_GRP11_GRVWA8_GRVWB8_GS UAMB_GLS0_HPLR0_ISA1201_ICIW0_IU1_K1_LDSTI0_LBSPPA512_LBSPPB0_LBSPPMXSA0_LBSPPMXSB0 _LBSPPM0_LPA16_LPB0_LPMXSA0_LPMXSB0_LPM0_LRVW8_LWPMn1_MIAV1_MIWT1_1_MXLIBL_MXSFN5_M 040_MGRIPM1_NTN1_NTA0 NTB0_NTC0_NTD0_NTE0_NTMXSA0_NTMXSB0_NTM0_NTWS0_NVn1_NVA0_NVBO _NVC0_NVD0_NVE0_NVMXSA0_NVMXSB0_NVM0_NVWS0_NEPBS0_NLCA1_NLCB16_ONLL0_PAP0_PGL0_PGR1 _PLR1_PKA0_SGROB0_SIA3_SS0_SPO0_SRVW0_SSO0_SVW8_SK0_SKFTR0_SKFDP00_SKXCCM0_SNLL0_SI P1_SGRO0_TDMI0_TDMIM0_TDMS0_TIN0_Thn1_THA0_THB0_THC0_THD0_THE0_THMXSA0_THMXSB0_THM0 _THWS0_TLDS1_TLDSM1_ULSGRO0_USL1_USLMX0_UIOFGRO0_UPLRP0_USFGR0n1_USI0_VSn1_VWA1_VWB 1_WSGRA0_WSGRB0_WS32_WG16_4_1.kd	6 / 6	0.176500	0.176667
drafter / Drafter __amd_rocclr_copyBuffer.kd	6 / 6	0.002272	0.001806
drafter / Drafter __amd_rocclr_fillBufferAligned.kd	18 / 18	0.007077	0.006984
drafter / Drafter _cache_draft_logits_kernel.kd	6 / 6	0.002359	0.002392
drafter / Drafter _draft_head_int2.kd	6 / 6	0.852255	0.826423
drafter / Drafter _gemm_a8w8_blockscale_preshuffle_kernel_GROUP_K_128_GROUP_N_128_BLOCK_SIZE_M_16_BLO CK_SIZE_N_128_BLOCK_SIZE_K_128_GROUP_SIZE_M_8_NUM_KSPLIT_1_SPLITK_BLOCK_SIZE_17408_ EVEN_K_1_GRID_MN_40_cache_modifier_NONE.kd	30 / 30	0.729304	0.732058
drafter / Drafter _gemm_a8w8_blockscale_preshuffle_kernel_GROUP_K_128_GROUP_N_128_BLOCK_SIZE_M_16_BLO CK_SIZE_N_128_BLOCK_SIZE_K_128_GROUP_SIZE_M_8_NUM_KSPLIT_1_SPLITK_BLOCK_SIZE_25600_ EVEN_K_1_GRID_MN_40_cache_modifier_NONE.kd	6 / 6	0.219960	0.228853
drafter / Drafter _gemm_a8w8_blockscale_preshuffle_kernel_GROUP_K_128_GROUP_N_128_BLOCK_SIZE_M_16_BLO CK_SIZE_N_128_BLOCK_SIZE_K_128_GROUP_SIZE_M_8_NUM_KSPLIT_1_SPLITK_BLOCK_SIZE_4096_E VEN_K_1_GRID_MN_40_cache_modifier_NONE.kd	30 / 30	0.201356	0.202723
drafter / Drafter _gemm_a8w8_blockscale_preshuffle_kernel_GROUP_K_128_GROUP_N_128_BLOCK_SIZE_M_16_BLO CK_SIZE_N_128_BLOCK_SIZE_K_128_GROUP_SIZE_M_8_NUM_KSPLIT_1_SPLITK_BLOCK_SIZE_5120_E VEN_K_1_GRID_MN_272_cache_modifier_NONE.kd	30 / 30	1.449473	1.447748
drafter / Drafter _gemm_a8w8_blockscale_preshuffle_kernel_GROUP_K_128_GROUP_N_128_BLOCK_SIZE_M_16_BLO CK_SIZE_N_128_BLOCK_SIZE_K_128_GROUP_SIZE_M_8_NUM_KSPLIT_1_SPLITK_BLOCK_SIZE_5120_E VEN_K_1_GRID_MN_48_cache_modifier_NONE.kd	30 / 30	0.272983	0.273570
drafter / Drafter _prepare_dflash_inputs_kernel.kd	6 / 6	0.007412	0.007879
drafter / Drafter _rerank_exact.kd	6 / 6	0.009099	0.009066
drafter / Drafter _selector_walk_kernel.kd	6 / 6	0.007892	0.007966
drafter / Drafter kernel_unified_attention.kd	30 / 30	1.828021	1.904776
drafter / Drafter reshape_and_cache_kernel_flash.kd	60 / 60	0.025323	0.027497
drafter / Drafter triton_per_fused_4.kd	6 / 6	0.001919	0.002006
drafter / Drafter triton_per_fused_9.kd	24 / 24	0.007656	0.007616

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Fixed ms per round
drafter / Drafter triton_per_fused__to_copy_abs_clamp_div_max_preshuffle_gemm_squeeze_view_0.kd	30 / 30	0.008442	0.009735
drafter / Drafter triton_per_fused__to_copy_abs_clamp_div_max_preshuffle_gemm_squeeze_view_2.kd	6 / 6	0.001932	0.002052
drafter / Drafter triton_per_fused__to_copy_abs_clamp_div_max_preshuffle_gemm_squeeze_view_4.kd	54 / 54	0.014324	0.015504
drafter / Drafter triton_per_fused__to_copy_abs_clamp_div_max_view_0.kd	6 / 6	0.003846	0.004206
drafter / Drafter triton_poi_fused_0.kd	6 / 6	0.002559	0.002586
drafter / Drafter triton_poi_fused_10.kd	24 / 24	0.007676	0.007963
drafter / Drafter triton_poi_fused_5.kd	6 / 6	0.002092	0.002266
drafter / Drafter triton_poi_fused__to_copy_clamp_div_mul_preshuffle_gemm_silu_slice_squeeze_view_7.kd	30 / 30	0.015335	0.015782
drafter / Drafter triton_poi_fused__to_copy_clamp_div_preshuffle_gemm_squeeze_view_1.kd	30 / 30	0.011568	0.010575
drafter / Drafter triton_poi_fused__to_copy_clamp_div_preshuffle_gemm_squeeze_view_3.kd	6 / 6	0.002032	0.002192
drafter / Drafter triton_poi_fused__to_copy_clamp_div_preshuffle_gemm_squeeze_view_5.kd	54 / 54	0.015618	0.016698
drafter / Drafter triton_poi_fused_add_arange_bitwise_and_constant_pad_nd_fused_add_rms_norm_ge_mul_select_slice_unsqueeze_view_3.kd	54 / 54	0.021045	0.018738
drafter / Drafter triton_poi_fused_add_arange_bitwise_and_constant_pad_nd_ge_mul_rms_norm_select_slice_unsqueeze_view_1.kd	6 / 6	0.002246	0.002372
drafter / Drafter triton_poi_fused_add_permute_unsqueeze_view_2.kd	6 / 6	0.001819	0.001886
drafter / Drafter triton_poi_fused_cat_expand_index_mul_slice_unsqueeze_view_1.kd	6 / 6	0.002579	0.002519
drafter / Drafter triton_red_fused__to_copy_abs_clamp_div_max_mul_preshuffle_gemm_silu_slice_squeeze_view_6.kd	30 / 30	0.013235	0.013422
drafter / Drafter triton_red_fused__to_copy_add_arange_bitwise_and_constant_pad_nd_fused_add_rms_norm_ge_mul_select_slice_unsqueeze_view_w4_gemm_2.kd	30 / 30	0.030395	0.031109
drafter / Drafter triton_red_fused__to_copy_add_arange_bitwise_and_constant_pad_nd_fused_add_rms_norm_ge_mul_select_slice_unsqueeze_view_w4_gemm_8.kd	24 / 24	0.025643	0.026496
drafter / Drafter triton_red_fused__to_copy_embedding_mul_rms_norm_w4_gemm_0.kd	6 / 6	0.003792	0.003859
drafter / Drafter triton_red_fused_add_arange_bitwise_and_constant_pad_nd_fused_add_rms_norm_ge_mul_select_slice_unsqueeze_view_8.kd	6 / 6	0.006652	0.006766

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Fixed ms per round
drafter / Drafter void at::native::(anonymous namespace)::CatArrayBatchedCopy_contig<at::native::(anonymous namespace)::OpaqueType<2u>, unsigned int, 2, 128, 1>(at::native::(anonymous namespace)::OpaqueType<2u>*, at::native::(anonymous namespace)::CatArrInputTensorMetadata<at::native::(anonymous namespace)::OpaqueType<2u>, unsigned int, 128, 1>, at::native::(anonymous namespace)::TensorSizeStride<unsigned int, 4u>, int, unsigned int) [clone .kd]	12 / 12	0.009118	0.009478
drafter / Drafter void at::native::_scatter_gather_elementwise_kernel<256, 4, at::native::_cuda_scatter_gather_internal_kernel<false, at::native::OpaqueType<4>, long>::operator()<at::native::TensorAssign>(at::TensorIterator&, long, long, long, at::native::TensorAssign const&)::{lambda(int)#1}>(int, at::native::_cuda_scatter_gather_internal_kernel<false, at::native::OpaqueType<4>, long>::operator()<at::native::TensorAssign>(at::TensorIterator&, long, long, long, at::native::TensorAssign const&)::{lambda(int)#1}) [clone .kd]	6 / 6	0.003566	0.003559
drafter / Drafter void at::native::_scatter_gather_elementwise_kernel<256, 4, at::native::_cuda_scatter_gather_internal_kernel<true, at::native::OpaqueType<2>, long>::operator()<at::native::TensorAssign>(at::TensorIterator&, long, long, long, at::native::TensorAssign const&)::{lambda(int)#1}>(int, at::native::_cuda_scatter_gather_internal_kernel<true, at::native::OpaqueType<2>, long>::operator()<at::native::TensorAssign>(at::TensorIterator&, long, long, long, at::native::TensorAssign const&)::{lambda(int)#1}) [clone .kd]	6 / 6	0.003006	0.002926
drafter / Drafter void at::native::bitonicSortKVInPlace<2, -1, 16, 16, c10::BFloat16, long, at::native::GTOp<c10::BFloat16, true>, unsigned int>(at::cuda::detail::TensorInfo<c10::BFloat16, unsigned int>, unsigned int, unsigned int, unsigned int, at::cuda::detail::TensorInfo<long, unsigned int>, unsigned int, at::native::GTOp<c10::BFloat16, true>) [clone .kd]	6 / 6	0.003219	0.003239
drafter / Drafter void at::native::elementwise_kernel_manual_unroll<128, 4, at::native::gpu_kernel_impl<at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda()#3}::operator()() const::{lambda()#4}::operator()() const:: {lambda(long)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()#3}::operator()() const::{lambda()#4}::operator()() const:: {lambda(long)#1} const&)::{lambda(int, bool)#1}>(int, at::native::gpu_kernel_impl<at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda()#3}::operator()() const::{lambda()#4}::operator()() const:: {lambda(long)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()#3}::operator()() const::{lambda()#4}::operator()() const:: {lambda(long)#1} const&)::{lambda(int, bool)#1}) [clone .kd]	6 / 6	0.004939	0.004986
drafter / Drafter void at::native::elementwise_kernel_manual_unroll<128, 4, at::native::gpu_kernel_impl_nocast<at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda()#3}::operator()() const::{lambda()#4}::operator()() const:: {lambda(long)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()#3}::operator()() const::{lambda()#4}::operator()() const:: {lambda(long)#1} const&)::{lambda(int, bool)#1}>(int, at::native::gpu_kernel_impl_nocast<at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda()#3}::operator()() const::{lambda()#4}::operator()() const:: {lambda(long)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()#3}::operator()() const::{lambda()#4}::operator()() const:: {lambda(long)#1} const&)::{lambda(int, bool)#1}) [clone .kd]	6 / 6	0.002472	0.002592

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Fixed ms per round
drafter / Drafter void at::native::elementwise_kernel_manual_unroll<128, 8, at::native::gpu_kernel_impl_nocast<at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda()}#3>::operator()() const:: {lambda()}#12>::operator()() const:: {lambda(c10::BFloat16)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()}#3>::operator()() const:: {lambda()}#12>::operator()() const:: {lambda(c10::BFloat16)#1} const&):: {lambda(int, bool)#1}>(int, at::native::gpu_kernel_impl_nocast<at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda()}#3>::operator()() const:: {lambda()}#12>::operator()() const:: {lambda(c10::BFloat16)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()}#3>::operator()() const:: {lambda()}#12>::operator()() const:: {lambda(c10::BFloat16)#1} const&):: {lambda(int, bool)#1}) [clone .kd]	6 / 6	0.003452	0.003606
drafter / Drafter void at::native::index_elementwise_kernel<128, 4, at::native::gpu_index_kernel<at::native::index_kernel_impl<at::native::OpaqueType<4 > >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1}>(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>, at::native::index_kernel_impl<at::native::OpaqueType<4 > >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1} const&, bool):: {lambda(int)#1}>(long, at::native::gpu_index_kernel<at::native::index_kernel_impl<at::native::OpaqueType<4 > >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1}>(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>, at::native::index_kernel_impl<at::native::OpaqueType<4 > >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1} const&, bool):: {lambda(int)#1}) [clone .kd]	6 / 6	0.003046	0.003112
drafter / Drafter void at::native::mbtopk::computeBlockDigitCounts<c10::BFloat16, unsigned int, unsigned int, 2>(at::cuda::detail::TensorInfo<c10::BFloat16 const, unsigned int>, unsigned int, unsigned int*, unsigned int, unsigned int, int, int, unsigned int, unsigned int, unsigned int*, short*) [clone .kd]	12 / 12	0.039078	0.038912
drafter / Drafter void at::native::mbtopk::computeBlockDigitCounts<float, unsigned int, unsigned int, 2>(at::cuda::detail::TensorInfo<float const, unsigned int>, unsigned int, unsigned int*, unsigned int, unsigned int, int, int, unsigned int, unsigned int, unsigned int*, short*) [clone .kd]	24 / 24	0.054229	0.056423
drafter / Drafter void at::native::mbtopk::computeBlockwiseWithinKCounts<unsigned int, c10::BFloat16>(unsigned int*, short*, unsigned int*, unsigned int, int, bool, unsigned int*, c10::BFloat16*, unsigned int*, unsigned int*, unsigned int*, unsigned int) [clone .kd]	12 / 12	0.022252	0.022331
drafter / Drafter void at::native::mbtopk::computeBlockwiseWithinKCounts<unsigned int, float>(unsigned int*, short*, unsigned int*, unsigned int, int, bool, unsigned int*, float*, unsigned int*, unsigned int*, unsigned int*, unsigned int) [clone .kd]	24 / 24	0.028783	0.028576
drafter / Drafter void at::native::mbtopk::fill<unsigned int, unsigned int>(unsigned int*, unsigned int, unsigned int) [clone .kd]	12 / 12	0.003045	0.002985
drafter / Drafter void at::native::mbtopk::gatherTopK<c10::BFloat16, unsigned int, 2>(at::cuda::detail::TensorInfo<c10::BFloat16 const, unsigned int>, unsigned int, unsigned int, bool, unsigned int, unsigned int, at::cuda::detail::TensorInfo<c10::BFloat16, unsigned int>, unsigned int, at::cuda::detail::TensorInfo<long, unsigned int>, unsigned int, unsigned int, unsigned int, c10::BFloat16*, unsigned int*, unsigned int*, unsigned int) [clone .kd]	6 / 6	0.023932	0.021492

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Fixed ms per round
drafter / Drafter void at::native::mbtopk::gatherTopK<float, unsigned int, 2>(at::cuda::detail::TensorInfo<float const, unsigned int>, unsigned int, unsigned int, bool, unsigned int, unsigned int, at::cuda::detail::TensorInfo<float, unsigned int>, unsigned int, at::cuda::detail::TensorInfo<long, unsigned int>, unsigned int, unsigned int, unsigned int, float*, unsigned int*, unsigned int*, unsigned int) [clone .kd]	6 / 6	0.010786	0.010859
drafter / Drafter void at::native::reduce_kernel<512, 1, at::native::ReduceOp<float, at::native::func_wrapper_t<float, at::native::sum_functor<float, float, float>::operator()(at::TensorIterator&)::lambda(float, float)#1>, unsigned int, float, 4, 4> >(at::native::ReduceOp<float, at::native::func_wrapper_t<float, at::native::sum_functor<float, float, float>::operator()(at::TensorIterator&):: lambda(float, float)#1>, unsigned int, float, 4, 4>) [clone .kd]	6 / 6	0.003932	0.003826
drafter / Drafter void at::native::vectorized_elementwise_kernel<4, at::native::CUDAFunctionOnSelf_add<long>, std::array<char*, 2ul> >(int, at::native::CUDAFunctionOnSelf_add<long>, std::array<char*, 2ul>) [clone .kd]	6 / 6	0.001799	0.001739
drafter / Drafter void at::native::vectorized_elementwise_kernel<4, at::native::bfloat16_copy_kernel_cuda(at::TensorIteratorBase&)::lambda(float)#1}, std::array<char*, 2ul> >(int, at::native::bfloat16_copy_kernel_cuda(at::TensorIteratorBase&)::lambda(float)#1}, std::array<char*, 2ul>) [clone .kd]	6 / 6	0.002506	0.002639
drafter / Drafter void at::native::vectorized_elementwise_kernel<4, at::native::bfloat16tofloat32_copy_kernel_cuda(at::TensorIteratorBase&):: lambda(c10::BFloat16)#1}, std::array<char*, 2ul> >(int, at::native::bfloat16tofloat32_copy_kernel_cuda(at::TensorIteratorBase&):: lambda(c10::BFloat16)#1}, std::array<char*, 2ul>) [clone .kd]	12 / 12	0.003751	0.003825
drafter / Drafter void at::native::vectorized_elementwise_kernel<8, at::native::FillFunctor<c10::BFloat16>, std::array<char*, 1ul> >(int, at::native::FillFunctor<c10::BFloat16>, std::array<char*, 1ul>) [clone .kd]	12 / 12	0.005245	0.005198
drafter / Drafter void at::native::vectorized_gather_kernel<16, long>(char*, char*, long*, int, long, long, long, long, bool) [clone .kd]	6 / 6	0.002119	0.002159
drafter / Drafter void at::native::warpMergeSortKVInPlace<2, -1, 128, 16, float, long, at::native::GT0p<float, true>, unsigned int, 32>(at::cuda::detail::TensorInfo<float, unsigned int>, unsigned int, unsigned int, unsigned int, at::cuda::detail::TensorInfo<long, unsigned int>, unsigned int, at::native::GT0p<float, true>, float) [clone .kd]	6 / 6	0.004912	0.004906
drafter / Drafter void r4d_gemm_w4a16_nt_m64_kernel<1, 1, false>(unsigned short const*, unsigned int const*, unsigned int const*, __hip_bfloat16*, int, int, int, int, int) [clone .kd]	6 / 6	0.004499	0.004599
drafter / Drafter void r4d_gemm_w4a16_nt_m64_kernel<1, 1, true>(unsigned short const*, unsigned int const*, unsigned int const*, __hip_bfloat16*, int, int, int, int, int) [clone .kd]	60 / 60	0.079510	0.081530
drafter / Drafter void vllm::rms_norm_kernel<c10::BFloat16, 8, 2, true>(c10::BFloat16*, c10::BFloat16 const*, long, long, long, long, long, c10::BFloat16 const*, long, float, int, int) [clone .kd]	6 / 6	0.002866	0.003039
drafter / Drafter void vllm::rms_norm_kernel<c10::BFloat16, 8, 4, true>(c10::BFloat16*, c10::BFloat16 const*, long, long, long, long, long, c10::BFloat16 const*, long, float, int, int) [clone .kd]	6 / 6	0.002732	0.002806

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Fixed ms per round
drafter / Drafter void vllm::rotary_embedding_kernel<c10::BFloat16, c10::BFloat16, true>(long const*, c10::BFloat16*, c10::BFloat16*, c10::BFloat16 const*, int, long, long, long, int, int, int, long, bool) [clone .kd]	6 / 6	0.002739	0.002786
target_body / Attention KV write reshape_and_cache_kernel_flash.kd	96 / 96	0.156952	0.046784
target_body / Attention Q/K normalization, RoPE and layout triton_poi_fused_1.kd	0 / 96	0.000000	0.023217
target_body / Attention Q/K normalization, RoPE and layout triton_poi_fused_6.kd	96 / 0	0.026617	0.000000
target_body / Attention Q/K normalization, RoPE and layout triton_poi_fused_8.kd	96 / 0	0.033151	0.000000
target_body / Attention Q/K normalization, RoPE and layout triton_poi_fused_add_cat_mul_slice_split_sub_unsqueeze_3.kd	0 / 96	0.000000	0.032391
target_body / Attention Q/K normalization, RoPE and layout triton_poi_fused_add_cat_mul_slice_split_sub_unsqueeze_4.kd	0 / 96	0.000000	0.031037
target_body / Attention Q/K normalization, RoPE and layout triton_poi_fused_arange_bitwise_and_eq_index_lt_remainder_select_split_where_2.kd	0 / 96	0.000000	0.028191
target_body / Attention Q/K normalization, RoPE and layout triton_red_fused_7.kd	96 / 0	0.159798	0.000000
target_body / Attention Q/K normalization, RoPE and layout void at::native::elementwise_kernel_manual_unroll<128, 8, at::native::gpu_kernel_impl_nocast<at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda()#3}::operator()() const:: {lambda()#12}::operator()() const:: {lambda(c10::BFloat16)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()#3}::operator()() const:: {lambda()#12}::operator()() const:: {lambda(c10::BFloat16)#1} const&):: {lambda(int, bool)#1}>(int, at::native::gpu_kernel_impl_nocast<at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()#3}::operator()() const:: {lambda()#12}::operator()() const:: {lambda(c10::BFloat16)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()#3}::operator()() const:: {lambda()#12}::operator()() const:: {lambda(c10::BFloat16)#1} const&):: {lambda(int, bool)#1}) [clone .kd]	0 / 96	0.000000	0.033677
target_body / Attention Q/K normalization, RoPE and layout void stock_m1_gemma_norm<false, 256, 32>(unsigned short const*, unsigned short const*, unsigned short const*, unsigned short*, unsigned short*, long, long, float, float*) [clone .kd]	0 / 96	0.000000	0.050924
target_body / Attention Q/K normalization, RoPE and layout void stock_m1_gemma_norm<false, 256, 64>(unsigned short const*, unsigned short const*, unsigned short const*, unsigned short*, unsigned short*, long, long, float, float*) [clone .kd]	0 / 96	0.000000	0.040317
target_body / Attention decode void qwen_stock_m1_shared_decode<4, 16, 256, 6, 16, 0, 3430971>(R4DArgs, int) [clone .kd]	0 / 96	0.000000	5.262466
target_body / Attention decode void r4d_attn_decode_kernel<3, 16, 256, 6, 16, 0, 3430971>(R4DArgs, int) [clone .kd]	96 / 0	4.112384	0.000000
target_body / Attention input activation FP8 quantization void vllm::dynamic_per_token_scaled_fp8_quant_kernel_strided<c10::BFloat16, c10::Float8_e4m3fn>(c10::Float8_e4m3fn*, float*, c10::BFloat16 const*, float const*, int, long, long) [clone .kd]	96 / 96	0.166878	0.042471
target_body / Attention input projection void radiance_mxfp4_fp8_gemm_decode<8, 128, 1, 1, true, true, true>(unsigned char const*, unsigned char const*, unsigned char const*, unsigned char const*, float const*, float*, int*, std::bfloat16_t*, int, int, int) [clone .kd]	96 / 96	1.103568	1.099129

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Fixed ms per round
target_body / Attention output activation FP8 quantization void vllm::dynamic_per_token_scaled_fp8_quant_kernel_strided<c10::BFloat16, c10::Float8_e4m3fn>(c10::Float8_e4m3fn*, float*, c10::BFloat16 const*, float const*, int, long, long) [clone .kd]	96 / 96	0.167458	0.046231
target_body / Attention output gating triton_poi_fused_mul_mxfp4_linear_sigmoid_view_0.kd	96 / 96	0.038584	0.032397
target_body / Attention output projection void radiance_mxfp4_fp8_gemm_decode<8, 128, 4, 1, true, true, true>(unsigned char const*, unsigned char const*, unsigned char const*, unsigned char const*, float const*, float*, int*, std::bfloat16_t*, int, int, int) [clone .kd]	96 / 96	0.561600	0.521093
target_body / Attention split-KV merge void qwen_stock_m1_shared_merge<256, 4, 1>(R4DArgs, int, int) [clone .kd]	0 / 96	0.000000	0.120825
target_body / Attention split-KV merge void r4d_attn_splitkv_combine_kernel<256, 4, 1>(R4DArgs, int, int) [clone .kd]	96 / 0	0.252525	0.000000
target_body / Embedding + first input normalization triton_poi_fused__to_copy_embedding_0.kd	0 / 6	0.000000	0.002572
target_body / Embedding + first input normalization triton_red_fused__to_copy_add_embedding_mxfp4_linear_rms_norm_0.kd	6 / 0	0.004019	0.000000
target_body / Embedding + first input normalization void stock_m1_gemma_norm<false, 5120, 512>(unsigned short const*, unsigned short const*, unsigned short const*, unsigned short*, unsigned short*, long, long, float, float*) [clone .kd]	0 / 6	0.000000	0.005126
target_body / Final normalization/layout triton_red_fused__to_copy_add_fused_add_rms_norm_3.kd	6 / 0	0.002619	0.000000
target_body / Final normalization/layout void stock_m1_gemma_norm<true, 5120, 512>(unsigned short const*, unsigned short const*, unsigned short const*, unsigned short*, unsigned short*, long, long, float, float*) [clone .kd]	0 / 6	0.000000	0.005759
target_body / GDN convolution _causal_conv1d_update_kernel.kd	0 / 288	0.000000	0.228132
target_body / GDN convolution void r4d_gdn_conv_update_kernel<1>(unsigned short const*, long, unsigned short const*, unsigned short const*, unsigned short*, long, long, long, int, int const*, long, int const*, unsigned short*, unsigned short*, unsigned short*, int const*, int, int, int) [clone .kd]	288 / 0	0.771508	0.000000
target_body / GDN input activation FP8 quantization void vllm::dynamic_per_token_scaled_fp8_quant_kernel_strided<c10::BFloat16, c10::Float8_e4m3fn>(c10::Float8_e4m3fn*, float*, c10::BFloat16 const*, float const*, int, long, long) [clone .kd]	288 / 288	0.487133	0.128053
target_body / GDN input projection void radiance_mxfp4_fp8_gemm_decode<8, 128, 1, 1, true, true, true>(unsigned char const*, unsigned char const*, unsigned char const*, unsigned char const*, float const*, float*, int*, std::bfloat16_t*, int, int, int) [clone .kd]	288 / 288	3.878766	3.874707
target_body / GDN layout/copies and buffer initialization triton_per_fused_1.kd	96 / 0	0.159812	0.000000
target_body / GDN layout/copies and buffer initialization triton_poi_fused_0.kd	96 / 0	0.033291	0.000000
target_body / GDN layout/copies and buffer initialization triton_poi_fused_add_1.kd	0 / 18	0.000000	0.004264
target_body / GDN layout/copies and buffer initialization triton_poi_fused_add_2.kd	0 / 12	0.000000	0.002725

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Fixed ms per round
target_body / GDN layout/copies and buffer initialization void at::native::elementwise_kernel_manual_unroll<128, 8, at::native::gpu_kernel_impl_nocast<at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda()}#3>::operator()() const::{lambda()}#12>::operator()() const::{lambda(c10::BFloat16)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()}#3>::operator()() const::{lambda()}#12>::operator()() const::{ {lambda(c10::BFloat16)#1} const&>::{lambda(int, bool)#1}>(int, at::native::gpu_kernel_impl_nocast<at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda()}#3>::operator()() const::{lambda()}#12>::operator()() const::{ {lambda(c10::BFloat16)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()}#3>::operator()() const::{lambda()}#12>::operator()() const::{ {lambda(c10::BFloat16)#1} const&>::{lambda(int, bool)#1}) [clone .kd]	288 / 288	0.462601	0.156493
target_body / GDN layout/copies and buffer initialization void at::native::vectorized_elementwise_kernel<4, at::native::bfloat16_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda(float)#1}, std::array<char*, 2ul> >(int, at::native::bfloat16_copy_kernel_cuda(at::TensorIteratorBase&)::{lambda(float)#1}, std::array<char*, 2ul>) [clone .kd]	0 / 288	0.000000	0.093412
target_body / GDN layout/copies and buffer initialization void at::native::vectorized_elementwise_kernel<8, at::native::FillFunctor<c10::BFloat16>, std::array<char*, 1ul> >(int, at::native::FillFunctor<c10::BFloat16>, std::array<char*, 1ul>) [clone .kd]	288 / 288	0.106299	0.074832
target_body / GDN output activation FP8 quantization void vllm::dynamic_per_token_scaled_fp8_quant_kernel_strided<c10::BFloat16, c10::Float8_e4m3fn>(c10::Float8_e4m3fn*, float*, c10::BFloat16 const*, float const*, int, long, long) [clone .kd]	288 / 288	0.254100	0.131386
target_body / GDN output gated normalization layer_norm_fwd_kernel.kd	0 / 288	0.000000	0.098186
target_body / GDN output gated normalization triton_per_fused__to_copy_mean_pow_view_0.kd	192 / 0	0.059768	0.000000
target_body / GDN output gated normalization triton_poi_fused__to_copy_add_mean_mul_mxfp4_linear_pow_rsqrtsilu_view_1.kd	192 / 0	0.307636	0.000000
target_body / GDN output gated normalization triton_poi_fused__to_copy_add_mean_mul_mxfp4_linear_pow_rsqrtsilu_view_2.kd	96 / 0	0.031951	0.000000
target_body / GDN output projection void radiance_mxfp4_fp8_gemm_decode<8, 128, 4, 1, true, true, true>(unsigned char const*, unsigned char const*, unsigned char const*, unsigned char const*, float const*, float*, int*, std::bfloat16_t*, int, int, int) [clone .kd]	288 / 288	1.896505	1.885747
target_body / GDN recurrence and gates stock_gdn_scan_kernel.kd	0 / 288	0.000000	1.217638
target_body / GDN recurrence and gates void r4d_gdn_recurrent_update_kernel<1, 0, 2>(unsigned short const*, unsigned short const*, unsigned short const*, void const*, void const*, long, float const*, float const*, float*, long, long, unsigned short*, int const*, int const*, long, int const*, unsigned short const*, float const*, float, int, int, int, float, float) [clone .kd]	288 / 0	1.150268	0.000000
target_body / Layer input residual/normalization triton_red_fused__to_copy_add_fused_add_rms_norm_mxfp4_linear_3.kd	90 / 0	0.039032	0.000000
target_body / Layer input residual/normalization triton_red_fused__to_copy_add_fused_add_rms_norm_mxfp4_linear_4.kd	192 / 0	0.082242	0.000000
target_body / Layer input residual/normalization triton_red_fused__to_copy_add_fused_add_rms_norm_mxfp4_linear_5.kd	96 / 0	0.059084	0.000000
target_body / Layer input residual/normalization void stock_m1_gemma_norm<true, 5120, 512>(unsigned short const*, unsigned short const*, unsigned short const*, unsigned short*, unsigned short*, long, long, float, float*) [clone .kd]	0 / 378	0.000000	0.357079

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Fixed ms per round
target_body / MLP SiLU and gating triton_poi_fused_mul_mxfp4_linear_silu_slice_0.kd	0 / 288	0.000000	0.133666
target_body / MLP SiLU and gating triton_poi_fused_mul_mxfp4_linear_silu_slice_1.kd	0 / 96	0.000000	0.045237
target_body / MLP SiLU and gating triton_poi_fused_mul_mxfp4_linear_silu_slice_2.kd	96 / 0	0.037757	0.000000
target_body / MLP SiLU and gating triton_poi_fused_mul_mxfp4_linear_silu_slice_3.kd	192 / 0	0.071995	0.000000
target_body / MLP SiLU and gating triton_poi_fused_mul_mxfp4_linear_silu_slice_4.kd	96 / 0	0.038051	0.000000
target_body / MLP down input FP8 quantization void vllm::dynamic_per_token_scaled_fp8_quant_kernel_strided<c10::BFloat16, c10::Float8_e4m3fn>(c10::Float8_e4m3fn*, float*, c10::BFloat16 const*, float const*, int, long, long) [clone .kd]	384 / 384	0.712899	0.298037
target_body / MLP down projection void radiance_mxfp4_fp8_gemm_decode<8, 128, 4, 1, true, true, true>(unsigned char const*, unsigned char const*, unsigned char const*, unsigned char const*, float const*, float*, int*, std::bfloat16_t*, int, int, int) [clone .kd]	384 / 384	5.173659	5.147158
target_body / MLP gate/up input FP8 quantization void vllm::dynamic_per_token_scaled_fp8_quant_kernel_strided<c10::BFloat16, c10::Float8_e4m3fn>(c10::Float8_e4m3fn*, float*, c10::BFloat16 const*, float const*, int, long, long) [clone .kd]	384 / 384	0.648425	0.167764
target_body / MLP gate/up projection void radiance_mxfp4_fp8_gemm_folded<2, true, true>(unsigned char const*, unsigned char const*, unsigned char const*, unsigned char const*, float const*, std::bfloat16_t*, int, int, int) [clone .kd]	384 / 384	24.336265	25.500468
target_body / Post-attention/GDN residual/normalization triton_red_fused__to_copy_add_fused_add_rms_norm_mxfp4_linear_1.kd	96 / 0	0.046751	0.000000
target_body / Post-attention/GDN residual/normalization triton_red_fused__to_copy_add_fused_add_rms_norm_mxfp4_linear_2.kd	192 / 0	0.069449	0.000000
target_body / Post-attention/GDN residual/normalization triton_red_fused__to_copy_add_fused_add_rms_norm_mxfp4_linear_3.kd	96 / 0	0.044531	0.000000
target_body / Post-attention/GDN residual/normalization void stock_m1_gemma_norm<true, 5120, 512>(unsigned short const*, unsigned short const*, unsigned short const*, unsigned short*, unsigned short*, long, long, float, float*) [clone .kd]	0 / 384	0.000000	0.353064
target_vocabulary_head / Full BF16 target head (anonymous namespace)::stock_m1_head_pair(__hip_bfloat16 const*, __hip_bfloat16 const*, __hip_bfloat16*) [clone .kd]	0 / 6	0.000000	4.134203
target_vocabulary_head / Full BF16 target head Cijk_Alik_Bljk_BBS_BH_Bias_HA_S_SAV_UserArgs_MT16x32x256_MI16x16x1_SN_LDSB0_AFC1_AG 0_AGGSUA0_AGNTAB0_AFEM1_AFEM1_ASEM1_CD1_1_CLR1_CLS0_CADS0_DTLA0_DTLB0_DTLMO_DTVA0_D TVB1_DTVMXSA0_DTVMSB0_DTVSM0_DPLB0_EPS1_EFLR0_EMLLn1_FDSI0_GRPm1_GRVWA8_GRVWB8_GS UAMB_GLS0_HPLR0_ISA1201_ICIW0_IU1_K1_LDSTI0_LBSPPA512_LBSPPB0_LBSPPMXSA0_LBSPPMXSB0 _LBSPPM0_LPA16_LPB0_LPMXSA0_LPMXSB0_LPM0_LRVW8_LWPMn1_MIAV1_MIWT1_1_MXLIBL_MXSFNS_M O40_MGRIPM1_NTn1_NTA0 NTB0_NTC0_NTD0_NTE0_NTMXSA0_NTMXSB0_NTM0_NTWS0_NVn1_NVA0_NVB0 _NVC0_NVD0_NVE0_NVMXSA0_NVMXSB0_NVM0_NVWS0_NEPBS0_NLCA1_NLCB16_ONLL0_PAP0_PGL0_PGR1 _PLR1_PKA0_SGROB0_SIA3_SS1_SPO0_SRVW0_SSO0_SVW1_SK0_SKFTR0_SKFDPO0_SKXCCM0_SNLL0_SI P1_SGRO0_TDMI0_TDMIM0_TDSM0_TIN0_Thn1_THA0_THB0_THC0_THD0_THE0_THMXSA0_THMXSB0_THM0 _THWS0_TLDS1_TLDSM1_ULSGRO0_USL1_USLMX0_UIOFGRO0_UPLRP0_USFGR0n1_USI0_VSn1_VWA1_VWB 1_WSGRA0_WSGRB0_WS32_WG16_4_1.kd	6 / 0	4.101219	0.000000
unattributed / Other GPU bookkeeping __amd_rocclr_copyBuffer.kd	172 / 172	0.111898	0.075011
unattributed / Other GPU bookkeeping __amd_rocclr_fillBufferAligned.kd	6 / 6	0.014906	0.002519

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Fixed ms per round
unattributed / Other GPU bookkeeping _combine_sampled_and_draft_tokens_kernel.kd	7 / 7	0.003459	0.003439
unattributed / Other GPU bookkeeping _compute_local_logits_stats_kernel.kd	6 / 6	0.028832	0.028913
unattributed / Other GPU bookkeeping _compute_slot_mappings_kernel.kd	7 / 7	0.003359	0.003439
unattributed / Other GPU bookkeeping _expand_idx_mapping_kernel.kd	7 / 7	0.002119	0.002192
unattributed / Other GPU bookkeeping _gather_block_tables_kernel.kd	7 / 7	0.005712	0.005586
unattributed / Other GPU bookkeeping _get_num_sampled_and_rejected_kernel.kd	6 / 6	0.014839	0.002699
unattributed / Other GPU bookkeeping _insert_resampled_kernel.kd	6 / 6	0.003452	0.003412
unattributed / Other GPU bookkeeping _post_update_kernel.kd	6 / 6	0.014692	0.005839
unattributed / Other GPU bookkeeping _prepare_pos_seq_lens_kernel.kd	7 / 7	0.002386	0.002486
unattributed / Other GPU bookkeeping _prepare_rope_positions_kernel.kd	7 / 7	0.003252	0.003432
unattributed / Other GPU bookkeeping _rejection_kernel.kd	6 / 6	0.019246	0.008532
unattributed / Other GPU bookkeeping _resample_kernel.kd	6 / 6	0.019092	0.015266
unattributed / Other GPU bookkeeping _scatter_num_accepted_kernel.kd	6 / 6	0.014586	0.002052
unattributed / Other GPU bookkeeping postprocess_mamba_fused_kernel.kd	6 / 6	0.016539	0.002566
unattributed / Other GPU bookkeeping precopy_mamba_align_fused_kernel.kd	7 / 7	0.003099	0.002992
unattributed / Other GPU bookkeeping preprocess_mamba_align_fused_kernel.kd	7 / 7	0.003085	0.002972
unattributed / Other GPU bookkeeping void (anonymous namespace)::elementwise_kernel_with_index<int, at::native::arange_cuda_out(c10::Scalar const&, c10::Scalar const&, c10::Scalar const&, at::Tensor&):={lambda()#1}::operator()() const::={lambda()#3}::operator()() const::={lambda(long)#1}>(int, at::native::arange_cuda_out(c10::Scalar const&, c10::Scalar const&, c10::Scalar const&, at::Tensor&):={lambda()#1}::operator()() const::={lambda()#3}::operator()() const::={lambda(long)#1}, function_traits<at::native::arange_cuda_out(c10::Scalar const&, c10::Scalar const&, c10::Scalar const&, at::Tensor&):={lambda()#1}::operator()() const::: {lambda()#3}::operator()() const::={lambda(long)#1}>::result_type*) [clone .kd]	42 / 42	0.012740	0.010306
unattributed / Other GPU bookkeeping void (anonymous namespace)::softmax_warp_forward<float, float, float, 6, false, false, 32>(float*, float const*, int, int, int, bool const*, int, bool) [clone .kd]	6 / 6	0.014786	0.002392
unattributed / Other GPU bookkeeping void at::native::_scatter_gather_elementwise_kernel<256, 4, at::native::_cuda_scatter_gather_internal_kernel<false, at::native::OpaqueType<4>, long>::operator()>(at::native::TensorAssign>(at::TensorIterator&, long, long, long, at::native::TensorAssign const&):={lambda(int)#1}>(int, at::native::_cuda_scatter_gather_internal_kernel<false, at::native::OpaqueType<4>, long>::operator()>(at::native::TensorAssign>(at::TensorIterator&, long, long, long, at::native::TensorAssign const&):={lambda(int)#1}) [clone .kd]	48 / 48	0.023805	0.015465

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Fixed ms per round
unattributed / Other GPU bookkeeping void at::native::_scatter_gather_elementwise_kernel<256, 4, at::native::_cuda_scatter_gather_internal_kernel<true, at::native::OpaqueType<4>, long>::operator()<at::native::TensorAssign>(at::TensorIterator&, long, long, long, at::native::TensorAssign const&)::{lambda(int)#1}>(int, at::native::_cuda_scatter_gather_internal_kernel<true, at::native::OpaqueType<4>, long>::operator()<at::native::TensorAssign>(at::TensorIterator&, long, long, long, at::native::TensorAssign const&)::{lambda(int)#1}) [clone .kd]	6 / 6	0.014932	0.003006
unattributed / Other GPU bookkeeping void at::native::elementwise_kernel_manual_unroll<128, 4, at::native::gpu_kernel_impl<at::native::direct_copy_kernel_cuda(at::TensorIteratorB ase&)::{lambda()#3}::operator()() const:: {lambda()#4}::operator()() const:: {lambda(long)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()#3}::operator()() const:: {lambda()#4}::operator()() const:: {lambda(long)#1} const&)::{lambda(int, bool)#1}>(int, at::native::gpu_kernel_impl<at::native::direct_copy_kernel_cuda(at::TensorIteratorB ase&)::{lambda()#3}::operator()() const:: {lambda()#4}::operator()() const:: {lambda(long)#1}>(at::TensorIteratorBase&, at::native::direct_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda()#3}::operator()() const:: {lambda()#4}::operator()() const:: {lambda(long)#1} const&)::{lambda(int, bool)#1}) [clone .kd]	48 / 48	0.027099	0.016612
unattributed / Other GPU bookkeeping void at::native::elementwise_kernel_manual_unroll<128, 4, at::native::gpu_kernel_impl_nocast<at::native::CUDataFunctor_add<int> >(at::TensorIteratorBase&, at::native::CUDataFunctor_add<int> const&)::{lambda(int, bool)#1}>(int, at::native::gpu_kernel_impl_nocast<at::native::CUDataFunctor_add<int> >(at::TensorIteratorBase&, at::native::CUDataFunctor_add<int> const&)::{lambda(int, bool)#1}) [clone .kd]	42 / 42	0.018740	0.016833
unattributed / Other GPU bookkeeping void at::native::elementwise_kernel_manual_unroll<128, 8, at::native::gpu_kernel_impl_nocast<at::native::(anonymous namespace)::CompareFunctor<float> >(at::TensorIteratorBase&, at::native::(anonymous namespace)::CompareFunctor<float> const&)::{lambda(int, bool)#1}>(int, at::native::gpu_kernel_impl_nocast<at::native::(anonymous namespace)::CompareFunctor<float> >(at::TensorIteratorBase&, at::native::(anonymous namespace)::CompareFunctor<float> const&)::{lambda(int, bool)#1}) [clone .kd]	18 / 18	0.031377	0.019244
unattributed / Other GPU bookkeeping void at::native::index_elementwise_kernel<128, 4, at::native::gpu_index_kernel<at::native::index_kernel_impl<at::native::OpaqueType<4 > >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1}>(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>, at::native::index_kernel_impl<at::native::OpaqueType<4> >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1} const&, bool)::{lambda(int)#1}>(long, at::native::gpu_index_kernel<at::native::index_kernel_impl<at::native::OpaqueType<4 > >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1}>(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>, at::native::index_kernel_impl<at::native::OpaqueType<4> >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1} const&, bool)::{lambda(int)#1}) [clone .kd]	109 / 109	0.064696	0.054935
unattributed / Other GPU bookkeeping void at::native::index_elementwise_kernel<128, 4, at::native::gpu_index_kernel<at::native::index_kernel_impl<at::native::OpaqueType<8 > >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1}>(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>, at::native::index_kernel_impl<at::native::OpaqueType<8> >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1} const&, bool)::{lambda(int)#1}>(long, at::native::gpu_index_kernel<at::native::index_kernel_impl<at::native::OpaqueType<8 > >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1}>(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>, at::native::index_kernel_impl<at::native::OpaqueType<8> >(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1} const&, bool)::{lambda(int)#1}) [clone .kd]	12 / 12	0.017905	0.005858

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Fixed ms per round
unattributed / Other GPU bookkeeping void at::native::index_elementwise_kernel<128, 4, at::native::gpu_index_kernel<at::native::index_put_kernel_impl<at::native::OpaqueType<8> >(at::TensorIterator&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1}>(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>, at::native::index_put_kernel_impl<at::native::OpaqueType<8> >(at::TensorIterator&, c10::ArrayRef<long>, c10::ArrayRef<long>)::{lambda(char*, char const*, long)#1} const&, bool)::{lambda(int)#1}>(long, at::native::gpu_index_kernel<at::native::index_put_kernel_impl<at::native::OpaqueType<8> >(at::TensorIterator&, c10::ArrayRef<long>, c10::ArrayRef<long>):: {lambda(char*, char const*, long)#1}>(at::TensorIteratorBase&, c10::ArrayRef<long>, c10::ArrayRef<long>, at::native::index_put_kernel_impl<at::native::OpaqueType<8> >(at::TensorIterator&, c10::ArrayRef<long>, c10::ArrayRef<long>)::{lambda(char*, char const*, long)#1} const&, bool)::{lambda(int)#1}) [clone .kd]	6 / 6	0.002979	0.002986
unattributed / Other GPU bookkeeping void at::native::mbtopk::computeBlockDigitCounts<float, unsigned int, unsigned int, 2>(at::cuda::detail::TensorInfo<float const, unsigned int>, unsigned int, unsigned int*, unsigned int, unsigned int, int, int, unsigned int, unsigned int, unsigned int*, short*) [clone .kd]	24 / 24	0.083669	0.049023
unattributed / Other GPU bookkeeping void at::native::mbtopk::computeBlockwiseWithinKCounts<unsigned int, float>(unsigned int*, short*, unsigned int*, unsigned int, int, bool, unsigned int*, float*, unsigned int*, unsigned int*, unsigned int*, unsigned int) [clone .kd]	24 / 24	0.079943	0.038196
unattributed / Other GPU bookkeeping void at::native::mbtopk::fill<unsigned int, unsigned int>(unsigned int*, unsigned int, unsigned int) [clone .kd]	6 / 6	0.001846	0.001592
unattributed / Other GPU bookkeeping void at::native::mbtopk::gatherTopK<float, unsigned int, 2>(at::cuda::detail::TensorInfo<float const, unsigned int>, unsigned int, unsigned int, bool, unsigned int, unsigned int, at::cuda::detail::TensorInfo<float, unsigned int>, unsigned int, at::cuda::detail::TensorInfo<long, unsigned int>, unsigned int, unsigned int, unsigned int, float*, unsigned int*, unsigned int*, unsigned int) [clone .kd]	6 / 6	0.034859	0.027033
unattributed / Other GPU bookkeeping void at::native::tensor_kernel_scan_innermost_dim<float, std::plus<float> >(float*, float const*, unsigned int, unsigned int, unsigned int, float, std::plus<float>) [clone .kd]	6 / 6	0.002592	0.002632
unattributed / Other GPU bookkeeping void at::native::unrolled_elementwise_kernel<at::native::CUDAFunction_add<int>, std::array<char*, 3ul>, 4, TrivialOffsetCalculator<2, unsigned int>, TrivialOffsetCalculator<1, unsigned int>, at::native::memory::LoadWithoutCast, at::native::memory::StoreWithoutCast>(int, at::native::CUDAFunction_add<int>, std::array<char*, 3ul>, TrivialOffsetCalculator<2, unsigned int>, TrivialOffsetCalculator<1, unsigned int>, at::native::memory::LoadWithoutCast, at::native::memory::StoreWithoutCast) [clone .kd]	42 / 42	0.012040	0.012033
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<16, at::native::BinaryFunction<bool, bool, bool, at::native::BitwiseOrFunction<bool> >, std::array<char*, 3ul> >(int, at::native::BinaryFunction<bool, bool, bool, at::native::BitwiseOrFunction<bool> >, std::array<char*, 3ul>) [clone .kd]	6 / 6	0.002799	0.002859
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<16, at::native::FillFunction<bool>, std::array<char*, 1ul> >(int, at::native::FillFunction<bool>, std::array<char*, 1ul>) [clone .kd]	7 / 7	0.002052	0.002319
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<16, at::native::bitwise_not_kernel_cuda(at::TensorIteratorBase&)::{lambda(bool)#1}, std::array<char*, 2ul> >(int, at::native::bitwise_not_kernel_cuda(at::TensorIteratorBase&)::{lambda(bool)#1}, std::array<char*, 2ul>) [clone .kd]	6 / 6	0.014859	0.002439

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Fixed ms per round
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::launch_clamp_scalar(at::TensorIteratorBase&, c10::Scalar, c10::Scalar, at::native::detail::ClampLimits)::{lambda()}#1::operator()() const:: {lambda()}#3::operator()() const:: {lambda(int)#1}, std::array<char*, 2ul> >(int, at::native::(anonymous namespace)::launch_clamp_scalar(at::TensorIteratorBase&, c10::Scalar, c10::Scalar, at::native::detail::ClampLimits):: {lambda()}#1::operator()() const:: {lambda()}#3::operator()() const:: {lambda(int)#1}, std::array<char*, 2ul>) [clone .kd]	42 / 42	0.012026	0.013993
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::launch_clamp_scalar(at::TensorIteratorBase&, c10::Scalar, c10::Scalar, at::native::detail::ClampLimits)::{lambda()}#1::operator()() const:: {lambda()}#4::operator()() const:: {lambda(long)#1}, std::array<char*, 2ul> >(int, at::native::(anonymous namespace)::launch_clamp_scalar(at::TensorIteratorBase&, c10::Scalar, c10::Scalar, at::native::detail::ClampLimits):: {lambda()}#1::operator()() const:: {lambda()}#4::operator()() const:: {lambda(long)#1}, std::array<char*, 2ul>) [clone .kd]	6 / 6	0.010586	0.001939
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::masked_fill_kernel(at::TensorIterator&, c10::Scalar const&):: {lambda()}#1::operator()() const:: {lambda()}#7::operator()() const:: {lambda(float, bool)#1}, std::array<char*, 3ul> >(int, at::native::(anonymous namespace)::masked_fill_kernel(at::TensorIterator&, c10::Scalar const&):: {lambda()}#1::operator()() const:: {lambda()}#7::operator()() const:: {lambda(float, bool)#1}, std::array<char*, 3ul>) [clone .kd]	6 / 6	0.023252	0.010086
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::where_kernel_impl(at::TensorIterator&)::{lambda()}#1::operator()() const:: {lambda()}#11::operator()() const:: {lambda(bool, float, float)#1}, std::array<char*, 4ul> >(int, at::native::(anonymous namespace)::where_kernel_impl(at::TensorIterator&)::{lambda()}#1::operator()() const:: {lambda()}#11::operator()() const:: {lambda(bool, float, float)#1}, std::array<char*, 4ul>) [clone .kd]	12 / 12	0.004878	0.004645
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<4, at::native::BUnaryFunctor<int, int, int, at::native::binary_internal::div_floor_kernel_cuda(at::TensorIteratorBase&):: {lambda()}#1::operator()() const:: {lambda()}#3::operator()() const:: {lambda(int, int)#1}>, std::array<char*, 2ul> >(int, at::native::BUnaryFunctor<int, int, int, at::native::binary_internal::div_floor_kernel_cuda(at::TensorIteratorBase&):: {lambda()}#1::operator()() const:: {lambda()}#3::operator()() const:: {lambda(int, int)#1}>, std::array<char*, 2ul>) [clone .kd]	42 / 42	0.016913	0.014973
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<4, at::native::CUDAFunctorOnSelf_add<int>, std::array<char*, 2ul> >(int, at::native::CUDAFunctorOnSelf_add<int>, std::array<char*, 2ul>) [clone .kd]	48 / 48	0.015999	0.018245
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<4, at::native::CUDAFunctorOnSelf_add<long>, std::array<char*, 2ul> >(int, at::native::CUDAFunctorOnSelf_add<long>, std::array<char*, 2ul>) [clone .kd]	6 / 6	0.006306	0.002019
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<4, at::native::CUDAFunctor_add<float>, std::array<char*, 3ul> >(int, at::native::CUDAFunctor_add<float>, std::array<char*, 3ul>) [clone .kd]	6 / 6	0.014879	0.001939
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<4, at::native::FillFunctor<float>, std::array<char*, 1ul> >(int, at::native::FillFunctor<float>, std::array<char*, 1ul>) [clone .kd]	12 / 12	0.029718	0.003565

Scope / stage / compiled kernel	Calls old / fixed	Old ms per round	Fixed ms per round
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<4, at::native::FillFunctor<int>, std::array<char*, 1ul> >(int, at::native::FillFunctor<int>, std::array<char*, 1ul>) [clone .kd]	7 / 7	0.001886	0.001952
unattributed / Other GPU bookkeeping void at::native::vectorized_elementwise_kernel<4, at::native::bfloat16tofloat32_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda(c10::BFloat16)#1}, std::array<char*, 2ul> >(int, at::native::bfloat16tofloat32_copy_kernel_cuda(at::TensorIteratorBase&):: {lambda(c10::BFloat16)#1}, std::array<char*, 2ul>) [clone .kd]	6 / 6	0.022673	0.009712
unattributed / Other GPU bookkeeping void at::native::vectorized_gather_kernel<16, long>(char*, char*, long*, int, long, long, long, long, bool) [clone .kd]	6 / 6	0.014479	0.002166
unattributed / Other GPU bookkeeping void at::native::warpMergeSortKVInPlace<2, -1, 128, 16, float, long, at::native::GTOp<float, true>, unsigned int, 32>(at::cuda::detail::TensorInfo<float, unsigned int>, unsigned int, unsigned int, unsigned int, at::cuda::detail::TensorInfo<long, unsigned int>, unsigned int, at::native::GTOp<float, true>, float) [clone .kd]	6 / 6	0.012773	0.004986

Prefill is outside this steady-decode timing table and has a separate causal/chunk-partition repair. CPU scheduling has no GPU-kernel duration here. The last row retains GPU bookkeeping outside the attributed model scopes; its individual kernels are listed without inventing missing semantic labels.

5. Brief unprofiled engine controls — 60K input, not 60K output

The approximately 80–87 tok/s figures are short controls on one 60,000-input- token Pi fixture. They are not results from generating 60,000 tokens.

Compiled configuration	Natural responses	Output tokens	Timed post-first seconds	Median round	Pooled post-first rate
Original, full BF16 target head	3	2,518	29.515	59.619 ms	85.212 tok/s
Correctness repair, before performance recovery	3	2,237	32.560	76.038 ms	68.612 tok/s
Repair after first three performance changes	3	2,237	26.625	60.985 ms	83.907 tok/s
Final repair, all four performance changes	3	2,295	27.270	59.891 ms	84.048 tok/s

The final control produced 743, 848 and 704 tokens at 86.790, 79.240 and 87.529 tok/s. Its middle response differed from the earlier repaired control starting at token offset 671. A repeat of the same build reproduced the earlier 743/790/704 lengths and all three output digests. The cause of this intermittent continuation difference is unresolved; the table retains the first complete measurement. The separate forced-token 10K agreement does not establish repeatability of these natural completions. See [brief-speed-controls.json](#).

These direct-engine controls exclude HTTP/Pi, tools, snapshot publication, cold prefill and warm-up. Rates pool tokens and time after the first output chunk; round latency is the median of the per-response steady-round medians. Throughput also depends on speculative acceptance, so it is not a pure kernel speed ratio. Clean timing passes do not record GPU profiles or copy activations.

Cost of preserving compiled BF16 rounding

Four fresh compiled runs used **before–after–after–before order**, with three natural responses per run on the same 60,000-token Pi prefix. Both settings use the final four performance repairs, full BF16 target head and piecewise GPU graphs. The only changed compiler setting is TORCHINDUCTOR_EMULATE_PRECISION_CASTS=0/1. The stage profiles in §4 and the preceding engine controls predate this rounding alignment.

Compiled setting	Natural responses	Output tokens	Median round	Committed tokens/round	Pooled post-first rate
Before rounding alignment	6	4,474	60.906 ms	5.288	84.123 tok/s
BF16 intermediate casts preserved	6	5,050	60.900 ms	5.211	83.070 tok/s

Preserving casts changed the median round by **-0.006 ms (-0.01%)** and the pooled token rate by **-1.25%**. Mean committed tokens per round changed by -1.45%. The per-response median rounds ranged from 60.245 to 61.415 ms before and 60.567 to 61.194 ms after.

Round time is effectively unchanged in this sample. The slightly lower token rate is consistent with fewer committed tokens per round. Every run used the same sampling settings and three seeds, but changed rounding produced different natural continuations. The nearest-even native RoPE correction applies to the **eager reference**; this measures the compiled half of the alignment. Startup, compilation, warm-up and cold prefill are excluded from the decode timings. All twelve natural completions are included; per-round timings and source identities are retained in [rounding-speed-abba.json](#).

Separate, earlier 60K-generated-token verify-head experiment

This earlier compiled experiment used 115 natural responses per method across 11 intact Pi request boundaries, with 57,008–65,527 input tokens. It predates the final D7 repairs and **does not measure the final corrected M8's long-run speed**. The head methods also have different numerical guarantees.

Earlier head method	Natural responses	Generated tokens	Timed post-first seconds	Post-first tok/s
Full BF16 reference head	115	60,598	946.230	63.920
INT2 block-8 / rerank-80	115	60,075	892.548	67.178
INT2 global-128 / BF16 rerank	115	60,348	892.266	67.506
INT2 global-256 / BF16 rerank	115	60,675	906.873	66.779

The first output chunks are excluded from both the rate numerator and timed denominator. See [earlier-60k-output-speed.json](#).

6. Repairs and upstream ownership

1. **GDN prefill**: raw per-token gates and an ordered transition avoid the diagnosed dependence on future gates, cumulative-gate rounding and chunk partition. Core implementation belongs with `libr4d`; adapters belong in `Radiance`.
2. **GDN decode**: match serial convolution arithmetic, gate precision, recurrent reductions, output rounding and retained-prefix state. Rejecting a speculative suffix must not make its state persistent. Packed QKV views remove copies.
3. **Gated normalization**: retain the serial row tile without serializing all eight rows. The vendored FLA dispatcher belongs in `vLLM`.
4. **Attention**: retain each query's causal range and serial reduction/split policy; share KV reads to recover performance. Core ownership is `libr4d`.
5. **Residual/QK/final normalization**: preserve the chosen finite-precision boundaries; retain FP32 residual sums in registers. Existing `vLLM` rounding PRs must be checked for overlap before proposing another repair.
6. **Full BF16 head**: interleave two arithmetic-preserving M4 groups in one HIP launch. This avoids the first repair's duplicated launch/concatenation overhead. The upstream skinny-GEMM implementation belongs in `vLLM`.
7. **Compiled integration**: install dispatch before tracing/capture, use opaque bindings where necessary, and validate startup-only fallbacks separately from real requests. Integration and conformance adapters belong in `Radiance`.

These are submission scopes, not a claim that the pinned adapters apply unchanged to current upstream main. The new ports require their own tests. Upstream links and submission state are maintained in `submissions.md`.

7. Execution-mode diagnosis and remaining limits

The eager/compiled mismatch was localized to intermediate BF16 casts and native RoPE multiplication. Aligning both produced exact eager/compiled **M8** output agreement on the controlled 320-position Pi replay. This is separate from the

compiled M1/M8 10K result in section 3; the full 10K eager/compiled comparison has not been repeated with the new rounding settings.

Controlled M8 results before and after rounding alignment

Each experiment uses the same 60,000-token Pi prefix and 320 saved continuation positions. Source, runtime, capacity and numerical settings are checked; the receipts explicitly admit the indicated intervention. Every table entry is a count out of 320.

Eager versus compiled experiment	Top-1	Top-10 set	Top-10 order	Top-20 set	Top-20 order	Full logit vectors exact
Baseline, before rounding alignment	319	146	14	66	0	0
Native SiLU in compiled only	318	154	21	66	0	0
Preserve compiled precision casts only	316	151	22	76	0	0
Preserve compiled casts + nearest-even native RoPE in eager	320	320	320	320	320	320

Only the final row also matches the complete final-prefill vector. Its retained scores and inclusive tie sets agree throughout. Both capture observers reproduce their uninstrumented controls. The [complete before/after boundary table](#) records exact captured inputs and outputs at all **465 matched boundaries**, across 320 decode and nine sampled prefill positions, under the [declared intervention admission](#).

These changes affect the diagnostic configuration, not the release used for the timing tables. Correcting native RoPE changes four top-1 predictions versus the original eager output ([eager-only comparison](#)). Agreement between modes is not evidence of improved task accuracy.

Intermediate BF16 casts

Before alignment, the first difference occurred in layer 0's MLP SiLU/gating, after identical GDN and gate/up projection outputs. The [isolated native replay](#) tested all 64 layers on common inputs: 20,480 decode layer-position evaluations and 576 sampled prefill evaluations. Each kernel reproduced its own captured output; none of the complete decode output rows agreed between modes.

```
eager:    BF16(BF16(SiLU(gate))) * up)
compiled: BF16(SiLU_FP32(gate)) * up)
```

The BF16-intermediate arithmetic oracle matched eager in all 20,480 decode evaluations. An independent FP32-intermediate calculation still differed from the generated compiled kernel at 17 individual values, so that formula alone is not a bit-exact specification of the compiled operation.

Attention sigmoid gating had the same cast-elision distinction: eager rounded the sigmoid to BF16 before multiplying; compiled retained it in FP32. A [common-input native pilot](#) reproduced each implementation's own output and found 0/8 equal output rows between modes. The receipt identifies and supersedes an earlier diagnostic argument-mapping error; the gate is compiled argument 1, not the attention-output argument 0.

TORCHINDUCTOR_EMULATE_PRECISION_CASTS=1 preserves these intermediate casts. The experiment checks the loaded compiler boolean as well as the environment setting. Cast preservation alone did not restore equality, as the table shows; it exposed the remaining RoPE difference after matching the first three layers.

Native RoPE multiplication

With compiler casts preserved, the first remaining discrepancy occurred in layer 3 after identical QKV projection and Q/K normalization. The [fine attention comparison](#) found:

Layer 3 boundary	Exact decode rows / 320	Exact sampled prefill rows / 9	Differing decode values
QKV projection	320	9	0
Query normalization	320	9	0
Key normalization	320	9	0
Query after RoPE	0	1	184,437

Layer 3 boundary	Exact decode rows / 320	Exact sampled prefill rows / 9	Differing decode values
Key after RoPE	0	1	30,984
Value input	320	9	0
Attention output	0	1	1,335,594
Gate input	320	9	0
Gated attention output	0	1	1,282,380

The later output differences inherit changed rotary inputs and state; this table does not establish separate attention-kernel defects. On the same normalized Q/K and selected BF16 cosine/sine coefficients, the [explicit arithmetic oracle](#) reproduced each mode exactly at all 320 decode and nine sampled prefill positions:

```
native eager:      RNE_BF16(RTZ_BF16(a*c) - RTZ_BF16(b*s))
compiled emulation: RNE_BF16(RNE_BF16(a*c) - RNE_BF16(b*s))
```

The second rotary half uses addition with the same product-rounding rules; the 192 non-rotary coordinates are unchanged. RTZ means round toward zero; RNE means round to nearest, ties to even. This isolates rotation arithmetic, not the separate selection of cosine/sine coefficients.

The [native replay](#) then compared the installed kernel with a private copy that changed only product rounding:

Native RoPE variant	Q and K exact versus original eager	Q and K exact versus compiled cast preservation
Original native multiplication	320/320 decode; 9/9 prefill	0/320 decode; 1/9 prefill
Explicit nearest-even multiplication	0/320 decode; 1/9 prefill	320/320 decode; 9/9 prefill

Both Q and K independently meet those counts. Position zero is the identity rotation. Input/coefficient immutability and injected one-bit errors are checked. Source, LLVM IR, AMD assembly and GPU-binary hashes bind the result to the executed implementations; the full-model worker also verifies that the modified kernel runs during the request.

The installed **Triton 3.7.1 / PyTorch 2.12.0+rocm7.14** build lowers BF16 multiplication to `llvm.amdgcn.fdot2.bf16.bf16`, emitted as `v_dot2_bf16_bf16` on the R9700. Its observed truncation is the defect already fixed by [Triton PR #11227](#), merged August 14, 2026. That fix replaces the special lowering with an FP32 multiply and explicit nearest-even conversion. The new evidence here is the defect's occurrence and causal isolation inside the captured Qwen RoPE path; the upstream repair is credited to its existing authors.

Historical cross-mode 10K results, before rounding alignment

The earlier D7-repaired eager M1/M8 pair agreed internally on all 10,000 positions, as did the final compiled pair. Comparing those eager and compiled runs gave the following results for both M1 and M8:

Prediction	Same set	Same order
Top-1	9,811/10,000 (98.11%)	9,811/10,000 (98.11%)
Top-10	4,845/10,000 (48.45%)	756/10,000 (7.56%)
Top-20	2,227/10,000 (22.27%)	4/10,000 (0.04%)

Every full-vector hash differed, including all 23 prefills; prefill top-1 agreed at 21/23 positions. These runs predate the rounding alignment above. They also differ in repair integration and sequence capacity (`max_num_seqs=1` for eager versus 2 for compiled), so they cannot isolate compilation as the cause. The [recovered historical comparison](#) preserves that scope. These numbers are historical measurements, not the result of the newly aligned configuration.

Remaining coverage limits

- The rounding-aligned eager/compiled result covers 320 M8 decode positions and the final prefill prediction. Full 10K eager/compiled and M1/M8 qualification under these changed settings remains outstanding. The brief compiled speed comparison is reported in §5; eager execution speed has not been measured.
- Captured activation equality does not certify uncaptured KV/GDN/convolution state, unpaired operations or independent MRoPE coefficient selection.

- The original compiled 10K replay covers the all-seven-accepted D7 path. Small operator tests exercise rejection boundaries and injected faults, but full model coverage of every rejection history, long-context range, concurrency, snapshot restore and hardware platform is not established.
- Neither experiment proves arbitrary-input equivalence to an independent mathematical reference, eliminates model-generated loops or measures improved coding accuracy. Evidence remains tied to its source, binary and configuration identities; changed implementations require qualification.

8. Evidence and reproduction

The `evidence/` directory contains aggregate comparisons and kernel profiles, with file SHA-256 values in `evidence-sha256.json`. Recompute the stage table, kernel accounting and aggregate assertions without GPU use:

```
uv run python reports/d7-rdna4-2026-09-17/build_report.py
```

The original Pi transcripts, token IDs, ranked token IDs and raw traces remain private. The aggregate measurements can be audited from the published receipts; independent end-to-end reproduction needs a public replacement workload. Synthetic operator regressions accompany the corresponding source submissions.

Pinned arithmetic repair SHA-256: 8deef78cd58b355d85d142caac58a88ea68ad576a31fe7769cc3db49b18235e9. Final performance bundle SHA-256: d5e74aedda327d5543a0f4a47aef80ec853c3eea5f9de49051b2d58ec109068. 10K summary seal: 740f52b749130d91ecb870153a02e8b28c232489c87997423146c2af2d5496f8.

References and attribution

- [Radiance](#), the qualified downstream stack.
- [libr4d](#), RDNA4 attention and recurrent kernels.
- [vLLM](#), including ROCm skinny GEMMs and vendored FLA.
- [Flash Linear Attention](#), serial GDN/normalization foundations.
- [vLLM batch invariance](#), related existing work.
- [PyTorch numerical accuracy](#), batched/reduction-order limitations.
- Existing normalization work: [vLLM #49639](#) and [#52243](#).

This contribution builds on those implementations. Batch invariance itself is not introduced by this report. Earlier DFlash RNG work in Radiance PR #8 backports [vLLM #54282](#) and is not presented as a new upstream discovery here.